Integrating Multiple IPs into a Mini-SoC using Verilog

Introduction

This lab session helps in understanding how to integrate multiple IPs into a mini-SoC using Verilog. This hands-on demo works with Verilator and GTKWave, and demonstrates simple IP interconnect, signal routing, and modularity, laying the foundation for full SoC design later.

Objective

To show how independent Verilog IP blocks (e.g., counter, ALU, MUX) can be instantiated and interconnected within a top-level SoC module, enabling students to understand modular IP reuse and signal-level integration.

Mini-Soc Block Diagram:

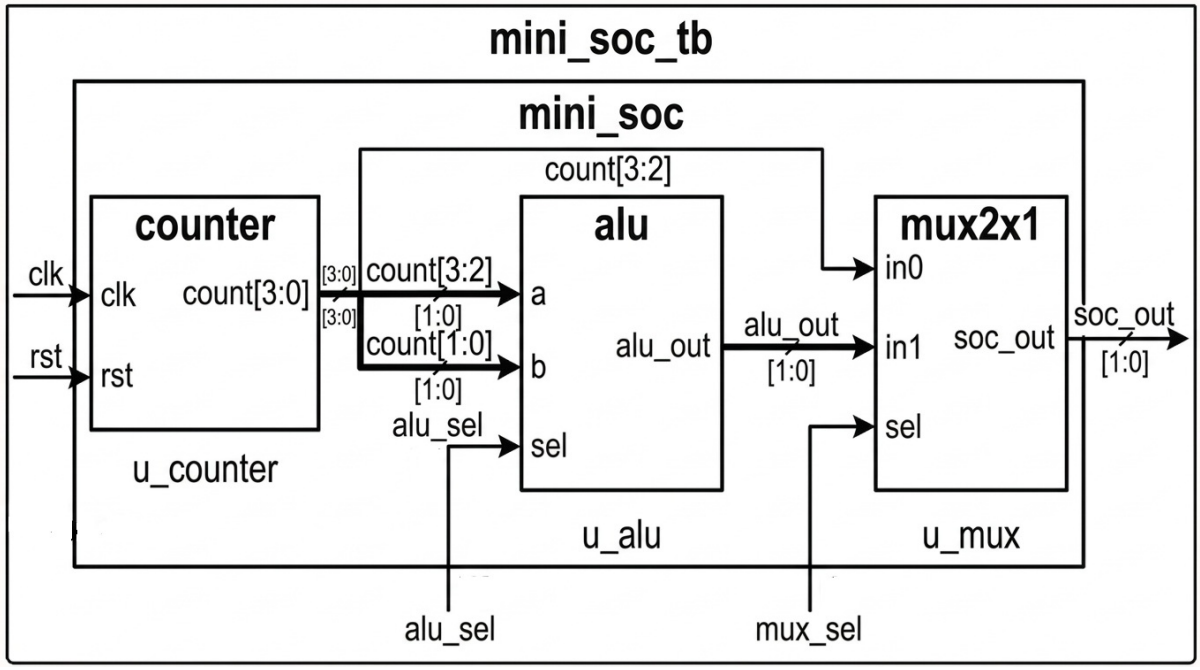


Fig: Mini-SoC Block Diagram

Procedure to create lab directory:

Create a directory mini_soc.

Design the lab:

Lab 1: 4-bit Counter IP

Design a synchronous 4-bit up-counter with reset, which serves as a reusable count source in the SoC.

This module acts as a simple hardware building block to demonstrate modularity.

Create the design file for counter IP : counter.v

```
// 4-bit synchronous counter
module counter (
  input clk,
  input rst,
  output reg [3:0] count
);
  always @(posedge clk) begin
    if (rst)
      count <= 0;
    else
      count <= count + 1;
  end
endmodule
```

Lab 2: 2-bit ALU IP (Add / Subtract)

Implement a 2-bit Arithmetic Logic Unit that performs either addition or subtraction based on a select input.

This IP demonstrates functional modularity and simple arithmetic operations in digital design.

Create the file for ALU IP : `alu.v`

```
// Simple 2-bit ALU with add and subtract
module alu (
  input [1:0] a,
  input [1:0] b,
  input sel,              // 0: ADD, 1: SUB
  output reg [1:0] result
);
  always @(*) begin
    case (sel)
      1'b0: result = a + b;
      1'b1: result = a - b;
    endcase
  end
endmodule
```

Lab 3: 2x1 Multiplexer IP

Create a 2:1 multiplexer that selects between two 2-bit input signals, forwarding the selected signal to the output.

This module provides simple data-path control, which is essential for system flexibility.

Create the file for Multiplexer IP: `mux2x1.v`

```
// 2-bit 2:1 MUX
module mux2x1 (
  input [1:0] in0,
  input [1:0] in1,
  input sel,
  output [1:0] out
);
  assign out = sel ? in1 : in0;
endmodule
```

Lab 4: SoC Top-Level Module

Integrate the counter, ALU, and MUX IP cores into a single top-level SoC module with clear interconnections.

This top-level module encapsulates the entire mini-SoC behavior and demonstrates full modular integration.

Create the file for SoC Top : `mini_soc.v`

```
// Integrates counter, ALU, and MUX
module mini_soc (
  input clk,
  input rst,
  input alu_sel,
  input mux_sel,
  output [1:0] soc_out
);
  wire [3:0] count;
  wire [1:0] alu_out;
  // Instantiate IP blocks
  counter u_counter (.clk(clk), .rst(rst),.count(count));
  alu u_alu (.a(count[3:2]),.b(count[1:0]),.sel(alu_sel),
.result(alu_out));
  mux2x1 u_mux (.in0(count[3:2]), .in1(alu_out),.sel(mux_sel),
.out(soc_out));
endmodule
```

Lab 5: Testbench for SoC

Develop a testbench that provides stimulus for clock, reset, and control signals to the SoC, then records outputs.

This environment enables automated functional verification and waveform analysis.

Create the testbench for SoC: `mini_soc_tb.v`

```
module mini_soc_tb;
  reg clk = 0;
```

```
  reg rst = 1;
  reg alu_sel = 0;
  reg mux_sel = 0;
  wire [1:0] soc_out;
  always #5 clk = ~clk;
  mini_soc dut (
    .clk(clk),
    .rst(rst),
    .alu_sel(alu_sel),
    .mux_sel(mux_sel),
    .soc_out(soc_out)
  );
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, mini_soc_tb);
    // Reset release
    #10 rst = 0;
    // Stimulate ALU and MUX selections
    #20 alu_sel = 1;
    #30 mux_sel = 1;
    #40 alu_sel = 0;
    #20 mux_sel = 0;
    #50 $finish;
  end
endmodule
```

Simulation Script

Provide an automated script for compiling, simulating, and visualizing waveforms using Verilator and GTKWave.

This script streamlines the simulation workflow and makes debugging efficient.

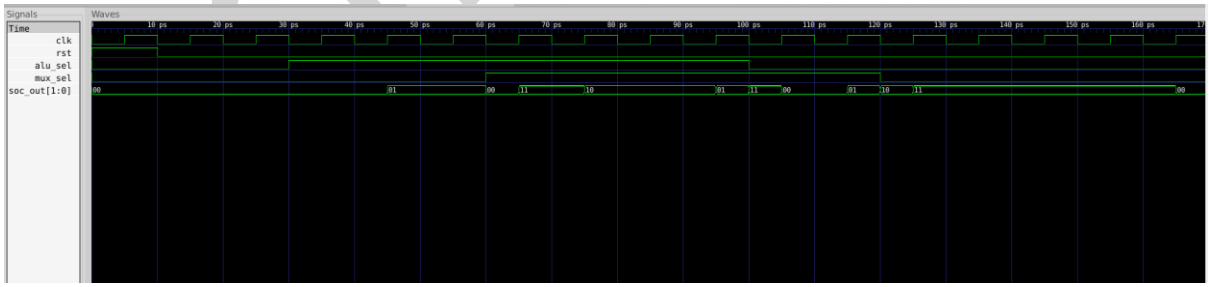
a) **Create a file for script : run.sh**

```
#!/bin/bash
# Use Verilator to compile and generate simulation files
verilator --binary -j 0 -Wall \
  counter.v alu.v mux2x1.v mini_soc.v mini_soc_tb.v \
  --top mini_soc_tb --timing --CFLAGS "-std=c++20" --trace
# Step 2: Enter the object directory
cd obj_dir || { echo "obj_dir not found"; exit 1; }
# Step 3: Compile simulation binary
make -f Vmini_soc_tb.mk Vmini_soc_tb || { echo "Compilation
failed"; exit 1; }
# Step 4: Run the simulation
./Vmini_soc_tb || { echo "Simulation failed"; exit 1; }
# Step 5: Launch GTKWave to view the waveform
gtkwave dump.vcd
```

b) **Make the script executable: `chmod +x run.sh`**

c) **Run the simulation and waveform viewer with : `./run.sh`**

GTKWave Waveform:



- **Counter Operation:** count increments on each clock cycle after reset is released.
- **ALU Behavior:** alu_out switches between addition and subtraction depending on the value of alu_sel.
- **MUX Output:** soc_out toggles between the counter's MSBs and the ALU output based on mux_sel.
- Control signal changes are reflected promptly in the waveform, enabling easy tracing of data flow and IP operation.

Conclusion

This lab session demonstrated how multiple, independent Verilog IP blocks can be integrated into a modular mini-SoC design. By connecting a 4-bit counter, a 2-bit ALU, and a 2x1 multiplexer within a top-level SoC module, you have explored fundamental signal routing and IP reuse concepts that underpin practical SoC development. Through simulation with Verilator and waveform analysis in GTKWave, you observed the dynamic interaction of control and data signals between the modules.